DS5102: Big Data Lab

Lecture-5: Introduction to Apache Hive
Post Lab Report

Muhamed Rizwan Mehaboob (142301026)
16th March 2026

Introduction

In this lab, we got hands-on with Apache Hive to see how it works with HDFS to store and analyze massive datasets. We looked at the differences between managed and external tables, and practiced pulling in data from both our local system and HDFS. We also tested out partitioning to see how it speeds up queries and observed Hive's schema-on-read approach in action.

Preliminaries: Setting up Hive

Hive Initialization

```
hadoop@machine1:~$ hive
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/home/hadoop/hive/lib/log4j-slf4j-impl-2.6.2.jar!/org/slf4j/imp
l/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/home/hadoop/hadoop-2.7.0/share/hadoop/common/lib/slf4j-log4j12
-1.7.10.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]

Logging initialized using configuration in jar:file:/home/hadoop/hive/lib/hive-common-2.3.9.jar!/
hive-log4j2.properties Async: true
Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider usin
g a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
hive>
```

Figure 1: Starting up the Hive shell

Database Creation and Selection

```
hive> CREATE DATABASE newdb;
OK
Time taken: 0.67 seconds
```

Figure 2: Creating a new database called newdb

```
hive> USE newdb;
OK
Time taken: 0.011 seconds
```

Figure 3: Switching over to use the newdb database

Loading Data: Local File System vs. HDFS

We started by loading files into both managed and external tables directly from the local file system. When using the `LOCAL` keyword, Hive copies the file from the local machine into HDFS. We then repeated this process for data already residing in HDFS. By removing the `LOCAL` keyword, Hive simply moves the existing HDFS file into its warehouse directory rather than creating a new copy.

Schema Evolution & Multiple Schemas

We also explored Hive's schema-on-read capability. Because Hive only verifies the data structure when a query is executed (not when data is inserted), the underlying raw data in HDFS remains completely untouched. This allows us to easily handle schema evolution—changing the schema of data that is already loaded. Furthermore, it allows us to create multiple tables with entirely different schemas that all point to the exact same underlying HDFS data file.

Task 1: Iris Data Analysis

```
hive> CREATE TABLE iris_avg_features AS
    > SELECT
    >     class,
    >     AVG(sepal_length) AS avg_sepal_length,
    >     AVG(sepal_width)  AS avg_sepal_width,
    >     AVG(petal_length) AS avg_petal_length,
    >     AVG(petal_width)  AS avg_petal_width
    > FROM iris_managed
    > GROUP BY class;
WARNING: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Cons
ider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
Query ID = hadoop_20260225102403_39ec007b-3d86-4121-9eff-cabde4784425
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1769087632970_0036, Tracking URL = http://machine1:8088/proxy/application_1769
087632970_0036/
Kill Command = /home/hadoop/hadoop-2.7.0/bin/hadoop job  -kill job_1769087632970_0036
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-02-25 10:24:10,299 Stage-1 map = 0%,  reduce = 0%
2026-02-25 10:24:16,604 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 1.41 sec
2026-02-25 10:24:22,925 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 3.26 sec
MapReduce Total cumulative CPU time: 3 seconds 260 msec
Ended Job = job_1769087632970_0036
Moving data to directory hdfs://machine1:9000/user/hive/warehouse/newdb.db/iris_avg_features
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 3.26 sec   HDFS Read: 15630 HDFS Write: 344 SU
CCESS
Total MapReduce CPU Time Spent: 3 seconds 260 msec
OK
Time taken: 21.847 seconds
```

Figure 4: Creating a new table called iris_avg_features

```
hive> SELECT * FROM IRIS_AVG_FEATURES;
OK
Iris-setosa     5.006000003814697       3.41800000667572        1.4639999961853027      0.24400000482797624
Iris-versicolor 5.935999975204468       2.770000009536743       4.259999980926514       1.3259999918937684
Iris-virginica  6.588000001907349       2.9739999914169313      5.551999988555909       2.0259999775886537
Time taken: 0.187 seconds, Fetched: 3 row(s)
hive>
```

Figure 5: The average feature values calculated for each Iris class

Using HiveQL, we analyzed the Iris dataset to determine the average sepal length, sepal width, petal length, and petal width for each of the three flower species. As required by the lab, the results of these aggregate calculations were successfully stored into a separate, newly created table called `iris_avg_features`.

Task 2: Movie Rating Data Analysis

Loading u.data into HDFS

```
hadoop@machine1:~$ hdfs dfs -put /home/hadoop/Downloads/Lecture5/u.data /Lecture5/Input
hadoop@machine1:~$ hdfs dfs -ls /Lecture5/Input/
Found 2 items
-rw-r--r--   2 hadoop supergroup       4606 2026-02-25 10:10 /Lecture5/Input/iris.csv
-rw-r--r--   2 hadoop supergroup    1979173 2026-02-25 10:45 /Lecture5/Input/u.data
```

Figure 6: Pushing the u.data file into HDFS and verifying it got there

Creating and Loading Table tb1

```
hive> CREATE TABLE tb1 (
    > user_id INT
    > , movie_id INT,
    > rating INT,
    > ts BIGINT
    > )
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY '\t'
    > STORED AS TEXTFILE;
OK
Time taken: 5.632 seconds
```

Figure 7: Setting up the tb1 table

```
hive> LOAD DATA INPATH '/Lecture5/Input/u.data'
    > into TABLE tb1;
Loading data to table default.tb1
OK
Time taken: 1.17 seconds
```

Figure 8: Dumping the movie dataset into tb1

First, the `ml-100k/u.data` dataset was loaded into a standard Hive table named `tb1`. We then queried this table to identify the top 5 movies strictly based on the total number of ratings they received.

Top 5 Movies by Number of Ratings

```
hive> SELECT
    >     movie_id,
    >     COUNT(*) AS total_ratings
    > FROM tb1
    > GROUP BY movie_id
    > ORDER BY total_ratings DESC
    > LIMIT 5;
WARNING: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
Query ID = hadoop_20260225105210_cd87f993-bbad-4b1b-8679-fe4d75c22bf0
Total jobs = 2
Launching Job 1 out of 2
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1769087632970_0037, Tracking URL = http://machine1:8088/proxy/application_1769087632970_0037/
Kill Command = /home/hadoop/hadoop-2.7.0/bin/hadoop job  -kill job_1769087632970_0037
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-02-25 10:52:19,296 Stage-1 map = 0%,  reduce = 0%
2026-02-25 10:52:24,690 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 2.13 sec
2026-02-25 10:52:30,994 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 4.03 sec
MapReduce Total cumulative CPU time: 4 seconds 30 msec
Ended Job = job_1769087632970_0037
Launching Job 2 out of 2
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1769087632970_0038, Tracking URL = http://machine1:8088/proxy/application_1769087632970_0038/
Kill Command = /home/hadoop/hadoop-2.7.0/bin/hadoop job  -kill job_1769087632970_0038
Hadoop job information for Stage-2: number of mappers: 1; number of reducers: 1
2026-02-25 10:52:42,507 Stage-2 map = 0%,  reduce = 0%
2026-02-25 10:52:47,719 Stage-2 map = 100%,  reduce = 0%, Cumulative CPU 1.46 sec
2026-02-25 10:52:55,220 Stage-2 map = 100%,  reduce = 100%, Cumulative CPU 3.61 sec
MapReduce Total cumulative CPU time: 3 seconds 610 msec
Ended Job = job_1769087632970_0038
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 4.03 sec   HDFS Read: 1986489 HDFS Write: 35691 SUCCESS
Stage-Stage-2: Map: 1  Reduce: 1   Cumulative CPU: 3.61 sec   HDFS Read: 41258 HDFS Write: 186 SUCCESS
Total MapReduce CPU Time Spent: 7 seconds 640 msec
OK
50      583
258     509
100     508
181     507
294     485
Time taken: 45.478 seconds, Fetched: 5 row(s)
```

Figure 9: Finding out which five movies had the most ratings

Enabling Dynamic Partition

```
hive> SET hive.exec.dynamic.partition.mode=nonstrict;
hive> SET hive.exec.max.dynamic.partitions=2000;
hive> SET hive.exec.max.dynamic.partitions.pernode=2000;
```

Figure 10: Tweaking Hive settings to allow dynamic partitioning

Creating Partitioned Table tb2

```
hive> CREATE TABLE tb2 (
    >     movie_id INT,
    >     rating INT,
    >     ts BIGINT
    > )
    > PARTITIONED BY (user_id INT)
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY '\t'
    > STORED AS TEXTFILE;
OK
Time taken: 0.113 seconds
```

Figure 11: Creating tb2, structured to partition data by user_id

```
hive> INSERT INTO TABLE tb2 PARTITION (user_id)
    > SELECT movie_id, rating, ts, user_id
    > FROM tb1;
WARNING: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider
 using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
Query ID = hadoop_20260225105800_e268fc03-0f44-474a-9db5-dc47a19faf49
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks is set to 0 since there's no reduce operator
Starting Job = job_1769087632970_0039, Tracking URL = http://machine1:8088/proxy/application_17690876
32970_0039/
Kill Command = /home/hadoop/hadoop-2.7.0/bin/hadoop job  -kill job_1769087632970_0039
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 0
2026-02-25 10:58:05,798 Stage-1 map = 0%,  reduce = 0%
2026-02-25 10:58:19,508 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 9.72 sec
MapReduce Total cumulative CPU time: 15 seconds 430 msec
Ended Job = job_1769087632970_0039
Stage-4 is selected by condition resolver.
Stage-3 is filtered out by condition resolver.
Stage-5 is filtered out by condition resolver.
Moving data to directory hdfs://machine1:9000/user/hive/warehouse/tb2/.hive-staging_hive_2026-02-25_1
0-58-00_260_2982648037727929396-1/-ext-10000
Loading data to table default.tb2 partition (user_id=null)

Loaded : 943/943 partitions.
         Time taken to load dynamic partitions: 36.15 seconds
         Time taken for adding to write entity : 0.083 seconds
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1   Cumulative CPU: 15.43 sec   HDFS Read: 1983319 HDFS Write: 1632717 SUCCESS
Total MapReduce CPU Time Spent: 15 seconds 430 msec
OK
Time taken: 84.034 seconds
```

Figure 12: Moving the data over into the partitioned tb2 table

```
hive> DESCRIBE tb2;
OK
movie_id                int
rating                  int
ts                      bigint
user_id                 int

# Partition Information
# col_name              data_type               comment

user_id                 int
Time taken: 1.09 seconds, Fetched: 9 row(s)
```

Figure 13: Checking out the schema for tb2

Next, we created a dynamically partitioned table (`tb2`) organized by `user_id`. We queried both `tb1` and `tb2` to calculate the average rating given by User 16.

Average Rating of User 16

```
hive> SELECT AVG(rating)
    > FROM tb1
    > WHERE user_id = 16;
WARNING: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider
 using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
Query ID = hadoop_20260225110029_4a535ce7-2700-4a9c-be26-1fb67579b987
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1769087632970_0040, Tracking URL = http://machine1:8088/proxy/application_17690876
32970_0040/
Kill Command = /home/hadoop/hadoop-2.7.0/bin/hadoop job  -kill job_1769087632970_0040
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-02-25 11:00:35,971 Stage-1 map = 0%,  reduce = 0%
2026-02-25 11:00:41,177 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 2.58 sec
2026-02-25 11:00:47,394 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 4.36 sec
MapReduce Total cumulative CPU time: 4 seconds 360 msec
Ended Job = job_1769087632970_0040
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 4.36 sec   HDFS Read: 1988337 HDFS Write: 117 SUCC
ESS
Total MapReduce CPU Time Spent: 4 seconds 360 msec
OK
4.328571428571428
Time taken: 20.136 seconds, Fetched: 1 row(s)
```

Figure 14: Querying the unpartitioned tb1 for User 16's average rating

```
hive> SELECT AVG(rating)
    > FROM tb2
    > WHERE user_id = 16;
WARNING: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider
 using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
Query ID = hadoop_20260225110330_40b60dc1-8f3b-4ca4-9f40-ae4dcdb701a4
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1769087632970_0043, Tracking URL = http://machine1:8088/proxy/application_17690876
32970_0043/
Kill Command = /home/hadoop/hadoop-2.7.0/bin/hadoop job  -kill job_1769087632970_0043
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-02-25 11:03:37,235 Stage-1 map = 0%,  reduce = 0%
2026-02-25 11:03:42,502 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 1.3 sec
2026-02-25 11:03:48,705 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 2.94 sec
MapReduce Total cumulative CPU time: 2 seconds 940 msec
Ended Job = job_1769087632970_0043
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 2.94 sec   HDFS Read: 10798 HDFS Write: 117 SUCCES
S
Total MapReduce CPU Time Spent: 2 seconds 940 msec
OK
4.328571428571428
Time taken: 18.834 seconds, Fetched: 1 row(s)
```

Figure 15: Querying the partitioned tb2 for the exact same thing

Since `tb2` is grouped by user ID, Hive skips scanning the whole dataset and jumps straight to User 16's folder, saving a ton of time.

External Table Demonstration

```
hive> CREATE EXTERNAL TABLE iris_external (
    >     sepal_length FLOAT,
    >     sepal_width FLOAT,
    >     petal_length FLOAT,
    >     petal_width FLOAT,
    >     class STRING
    > )
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY ','
    > STORED AS TEXTFILE
    > LOCATION '/Lecture5/Input'
    > TBLPROPERTIES ("skip.header.line.count"="1");
OK
Time taken: 0.536 seconds
```

Figure 16: Creating an external table named iris_external

```
hive> DROP TABLE iris_external;
OK
Time taken: 0.759 seconds
```

Figure 17: Dropping the external table

```
hadoop@machine1:~$ hdfs dfs -ls /Lecture5/Input
Found 1 items
-rw-r--r--   2 hadoop supergroup       4606 2026-02-25 10:10 /Lecture5/Input/iris.csv
```

Figure 18: Confirming our raw data is still sitting pretty in HDFS

External tables are great because Hive only manages the metadata. If you accidentally (or intentionally) drop an external table, Hive deletes the table definition, but your actual data files stay completely safe in HDFS.

Utility of Partitioning and Bucketing in Hive

When dealing with massive datasets, scanning an entire table for every query is highly inefficient. Hive solves this using partitioning and bucketing.

Partitioning: This technique organizes table data into separate HDFS directories based on specific column values (like grouping by `user_id`). When querying a specific partition, Hive skips reading unnecessary data, which drastically cuts down query execution time.

Bucketing: This further breaks down data into a fixed number of files (buckets) using a hash function on a specific column. It is incredibly useful for quickly sampling data or optimizing queries that involve joining massive tables.

Schema-on-Write vs Schema-on-Read

Schema-on-Write (e.g., Traditional RDBMS)

- **Advantages:** Queries generally run faster because the data structure is enforced and validated before it is ever stored.
- **Disadvantages:** Loading data is a slower process because every record must be checked against the schema. Altering the schema later is restrictive and difficult.

Schema-on-Read (e.g., Apache Hive)

- **Advantages:** Data loading is exceptionally fast because raw files are simply moved into storage without upfront validation. It offers massive flexibility for handling semi-structured or constantly changing data formats.
- **Disadvantages:** Queries can be slightly slower because the schema must be applied on the fly during read time. Corrupted data won't be caught until a query is actively run.

Conclusion

This lab gave us practical experience with Apache Hive and showed how it teams up with HDFS. We created both managed and external tables, and ran some analysis on the Iris and MovieLens datasets using HiveQL. We saw firsthand how partitioning makes queries run much more efficiently by cutting down the amount of data scanned. Finally, we explored Hive's schema-on-read approach, which highlights how flexible big data tools can be when handling massive, unstructured files.